Software Development Project — G&G City · Lab Backlog Sprint 1

LAB P3: Command Verifier + Documentation — Abdulrahman Al-Ghamdi

---

Topic: Verifying Git Commands in a Real Terminal and Documenting the Story

Learning outcomes

By the end of this lab, students will be able to:

- Test a Git command safely in a throw-away folder
- Find which spellings Git really accepts
- Document verified results so others can trust them

🎯 Problem Scenario

If the game says a command is right but it fails in a real terminal, we teach the learner something wrong. You are the safety net.

- **P1 writes the commands, you verify them, then fill `verifiedBy` and `verifiedOn` in `lessons.json`**
- **You also write the station story so S1 knows what to draw**

Your files: `data/lessons.json (only verifiedBy / verifiedOn), docs/story.md`

Required Outputs

Build:

1. **Test every `accepted` command in a throw-away folder**
2. **Test every `commonMistakes` input and record what Git really does**
3. **Fill `verifiedBy` / `verifiedOn` and report differences to P1**
4. **Write the story of the four stations**

Code is given only for the hard parts. Parts marked 📝 **Note** are easy: follow the note and write them yourself. Test with **Live Server** in VS Code (`fetch` does not work when you double-click a file).

Model Answer:

Task 1 + 2: Testing in a throw-away folder (hard part)

Never test inside the project folder: `git init` there would create a repository inside our repository.

```
# PowerShell
cd $env:TEMP
mkdir gg-test; cd gg-test

git init          # expect: Initialized empty Git repository in ...
git init .        # run in a NEW empty folder, expect the same message
git int           # expect: git: 'int' is not a git command. The most similar command is
'init'

cd ..; Remove-Item -Recurse -Force gg-test   # clean up
```

Something to check carefully

> 📝 **Note (no code needed):** Try `Git init` (capital G) on Windows **and** on Mac/Linux. On Windows it may work because the file system ignores case, but the game rejects it to teach the correct habit. Write what happened on each system and tell P1, so the cat's message is accurate.

Task 3: Recording the result

> 📝 **Note (no code needed):** In `lessons.json` set `"verifiedBy": "P3"` and `"verifiedOn": "2026-11-10"` (the real date) for each lesson you checked. Only edit these two fields, then tell P1.

Task 4: The story

> 📝 **Note (no code needed):** In `docs/story.md`, one paragraph per station: where the cat is, what problem it has, which command solves it, and what changes on screen. Example: *The workshop — the cat's box is invisible to Git → `git init` → a .git sticker appears.*

Submission

```
git pull
git add data/lessons.json
git commit -m "توثيق التحقق من أوامر محطة الورشة"
git push
```